

Prompts & AI Workflow Documentation

Track 2 — Commercial Smart Facility & Sustainability Manager
KOHLEMITWPU AI Research Lab Program | Individual Submission

1. Overview

This document records how AI was used throughout the design and construction of this submission, and reproduces the exact prompt embedded in the running system that powers its auto-dispatch intelligence layer. AI was used in two distinct ways: (a) as a development assistant, to plan the architecture and write/test the code in this repository, and (b) as a runtime component of the shipped product itself, where a large language model converts raw sensor alerts into structured maintenance tickets. Both are documented below.

2. Development Workflow

The system was built conversationally with Claude (Anthropic) acting as a pair-programmer, working through the problem in three phases:

Phase	Focus	Key output
Day 1	Data + detection	Synthetic telemetry simulator, SQLite storage, and three rule-based detection functions (leak, hygiene threshold, sensor fault).
Day 2	Intelligence + UI	Streamlit operations dashboard; LLM-based ticket generation with a deterministic fallback for resilience.
Day 3	Packaging	This documentation, presentation deck, and video walkthrough.

2.1 Representative prompts used during development

- "I've selected Track 2. Guide me through everything from beginning to end, step by step, given I haven't started and need to submit by the 18th."
- "Give all the steps and all the coding required for Day 1 [telemetry simulator, SQLite storage, and the three detection rules] on VS Code in Python."
- "Build Day 2: the Streamlit dashboard and an AI-powered layer that turns each detected alert into an auto-generated maintenance ticket."
- "Does this need to be a hosted website?" / "How should I build the dashboard it's asking for?" — used to confirm scope before over-building.

Each response was tested (via automated smoke tests exercising the simulator, detection rules, ticket generation, and a headless render of the dashboard) before being accepted, rather than used as-is.

3. Runtime AI Component — Ticket Generation

The shipped application calls the Anthropic API (model: **claude-sonnet-4-6**) every time the detection engine raises an alert. The model is given fixed facility context plus the specific alert, and is instructed to return a structured JSON diagnosis. This is the core AI-driven component of the submission, implemented in `ticketing.py`.

3.1 Facility context provided to the model

Facility: International airport terminal, high footfall (approx. 40,000 passengers/day).
Restroom units are monitored by KOHLER smart sensors reporting water flow, flush events, occupancy, and a diagnostic heartbeat.
Maintenance teams available: PLUMBING, HOUSEKEEPING, SENSOR_TECH.
Water conservation and passenger experience are both priorities.

3.2 System prompt (verbatim)

You are the dispatch intelligence layer of a commercial smart facility management platform. You receive a raw sensor alert and convert it into a structured maintenance ticket for a facility operations team.

Reason about: how urgent this genuinely is given footfall and water waste, what the most likely physical cause is, and which team should handle it.

Respond with ONLY a valid JSON object, no preamble, no markdown fences, with exactly these keys:

```
{
  "title": "short ticket title, max 10 words",
  "priority_score": integer 1-100 where 100 is most urgent,
  "priority_label": one of "CRITICAL", "HIGH", "MEDIUM", "LOW",
  "likely_cause": "one or two sentences on the most probable physical cause",
  "recommended_action": "one or two sentences of concrete instruction
                        for the technician",
  "assigned_team": one of "PLUMBING", "HOUSEKEEPING", "SENSOR_TECH",
  "est_water_loss_lpd": estimated litres per day lost if unaddressed,
                      number, 0 if not a leak
}
```

3.3 Per-alert user prompt (template)

{facility_context}

INCOMING ALERT
Type: {alert_type}
Raw severity: {severity}
Location: {location}
Sensor ID: {sensor_id}
Timestamp: {timestamp}
Detail: {message}

Generate the maintenance ticket JSON.

3.4 Design rationale

- **Structured JSON output** — chosen over free text so the response can be parsed directly into the tickets table and rendered on the dashboard without further processing.

- **Facility context in the system prompt** — grounds the model's priority reasoning in real operational constraints (footfall, available teams) rather than judging alerts in isolation.
- **Deterministic fallback generator** — if the API key is missing or the call fails for any reason, a rules-based generator produces an equivalent ticket immediately. This was a deliberate reliability decision: a facility dispatch system cannot go silent because an external API had a bad moment. Every ticket records which path produced it (`generated_by: claude-llm` or `rules-fallback`) for auditability.
- **Response validation** — the application checks that all required keys are present and correctly typed before trusting an LLM response; anything malformed is silently treated as unavailable and routed to the fallback, rather than crashing the pipeline.

4. AI Tooling Summary

Use	Tool	Role
Development assistance	Claude (Anthropic), chat interface	Architecture planning, code generation, iterative testing/debugging across Day 1-3 of the build.
Runtime intelligence	Claude Sonnet 4.6, via Anthropic Messages API	Converts each detected alert into a structured, prioritized maintenance ticket with cause, action, and team assignment.
Runtime resilience	Deterministic rules engine (no external AI)	Fallback ticket generator ensuring uninterrupted dispatch if the API is unavailable.

Full source code, including the complete implementation referenced above, is available in the accompanying GitHub repository.